

FLEET: Token-Based Feature Extraction for Event Camera-based Reinforcement Learning

Tristan Gottwald[✉], Maximilian Schier[✉], Melanie Schaller[✉], and Bodo
Rosenhahn[✉]

Institute of Information Processing, L3S, Leibniz University Hannover, Germany
{gottwald,schier,schaller,rosenhahn}@tnt.uni-hannover.de

Abstract. Event cameras generate asynchronous, high-frequency data streams offering spatially sparse information at lower latency than traditional cameras. In principle, these properties should be ideal for the design of control policies. However, reinforcement learning research in this field remains limited as existing approaches fail to fully exploit the sensor’s properties. CNN-based methods negate the sensors benefits by aggregating events into sparse grids. This couples compute cost to sensor resolution and blurs the temporal information. Meanwhile, existing generative baselines rely on the availability of trajectory data to pretrain the model. We propose FLEET (Feature Learning from Events via Efficient Tokenization), a feature extractor that processes event sequences directly. Leveraging random Fourier features and cross-attention, our architecture compresses variable streams into fixed-size latent representations. This decouples inference cost of the feature extractor’s backbone from the sensor’s resolution, enabling end-to-end learning without auxiliary losses. We validate FLEET on a new, high-throughput benchmark. The results demonstrate that our sequence-based approach surpasses SOTA performance and exhibits superior robustness to variations in observation frequencies. Code: <https://github.com/tgottwald/FLEET>

Keywords: Reinforcement Learning · Event Camera · Feature Extractor · Autonomous Driving

1 Introduction

In recent years, event cameras have emerged as a transformative alternative to standard frame-based sensors [7, 17, 40, 47]. Unlike traditional cameras that capture snapshots of the world at fixed intervals, event cameras respond asynchronously to brightness changes at the pixel level [18]. This paradigm offers microsecond-latency and high dynamic range, making them theoretically ideal for time-critical applications [14, 61] while significantly reducing the data bandwidth required for transmission [18]. These properties are particularly advantageous for robotics [14, 15, 28, 32, 35] and autonomous driving [5, 6, 19, 33, 47], domains characterized by highly dynamic settings where hardware is constrained and low latency is essential for safety [18]. However, realizing this potential in re-

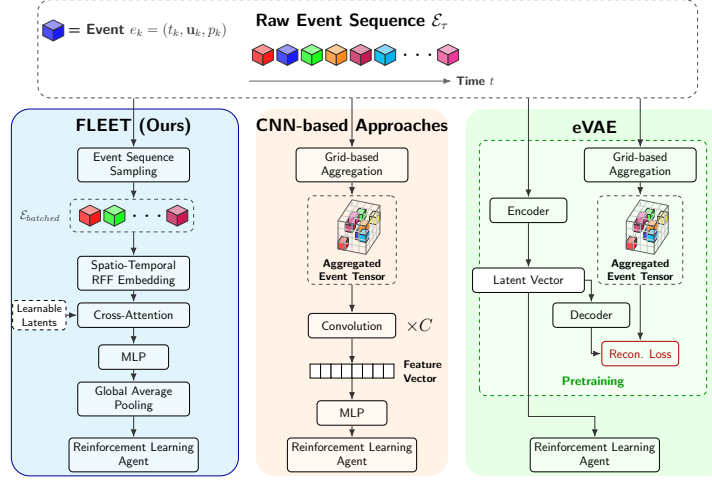


Fig. 1: Comparison of feature extraction architectures for event camera-based reinforcement learning. Our proposed FLEET architecture (left) processes raw sequence data directly, successfully preserving temporal and spatial resolution. Events are encoded using random Fourier features and mapped to a fixed set of learnable latent vectors using cross-attention. In contrast, standard CNN-based approaches (center) force events into extremely sparse aggregated tensor representations, leading to quantization errors and unnecessary processing overhead due to sparsity. Generative pre-training pipelines like eVAE (right) attempt to solve this but suffer from an objective mismatch: pretraining on an auxiliary reconstruction task forces the representation to encode visual fidelity rather than the features actually relevant to optimal control. Furthermore, these frozen, two-stage approaches can suffer significantly from distribution shifts if the offline pretraining data fails to capture the full scope of dynamics encountered during active deployment.

inforcement learning (RL) remains a significant challenge due to the sparse, asynchronous, and irregular structure of event data. Standard deep learning pipelines typically rely on convolutional neural networks (CNN) designed for dense, synchronous images [12, 34, 56]. To bridge this modality gap, existing approaches aggregate event streams into sparse spatio-temporal tensors [20, 54, 55, 63]. We argue that this is counterproductive for control: at standard control frequencies, aggregation introduces motion blur that destroys temporal precision, while at high frequencies, it generates computationally wasteful, sparse tensors where the compute cost scales with sensor resolution rather than information content. Alternative approaches employing representation learning, such as approaches based on variational autoencoders [53], attempt to compress this data into latent vectors. However, these methods rely on auxiliary reconstruction losses that prioritize visual fidelity over control-relevant features [58]. This creates a misalignment of objectives, where the encoder wastes capacity modeling irrelevant background dynamics rather than the task-critical physics required by the

agent. Furthermore, these two-stage pipelines can suffer from distribution shift if the data used during the pretraining is not representative of the data encountered during training. In conclusion, there is a distinct lack of investigation into lightweight, end-to-end trainable architectures that can process event streams directly within the RL loop.

To address these limitations, we propose FLEET (Feature Learning from Events via Efficient Tokenization), a tokenized, sequence-based processing pipeline as shown in Fig. 1. While the initial dimensionality reduction of the cross-attention module scales linearly with the number of input events, it maps the embedded events to a fixed-size set of latent tokens [26, 42]. This architecture ensures that the computational cost of all subsequent processing remains independent of the input resolution or event density. We leverage random Fourier features [39] to overcome the spectral bias of standard MLPs [50], enabling the network to capture high-frequency spatial details without explicit grid discretization.

The **main contributions** of this paper are as follows:

- **Spatial Resolution-Decoupled Feature Extractor:** We present a novel, end-to-end trainable architecture that processes raw event sequences using a Perceiver-style cross-attention mechanism. By querying the variable-length event stream with a fixed set of latent queries, we decouple the computational cost of the heavy processing block from the sensor resolution, enabling efficient scaling to sensors with higher spatial resolution.
- **Task-Aligned Representation Learning:** Unlike generative baselines, our architecture is trained purely on the RL reward signal without auxiliary reconstruction losses. We demonstrate that this yields task-aligned representations that generalize well to distribution shifts during deployment, as the model learns to attend solely to control-relevant dynamics.
- **Spectral Bias Mitigation:** We integrate random Fourier features to embed coordinate inputs. We show that this allows the network to resolve high-frequency spatial details essential for obstacle avoidance, which are often lost in grid aggregations.
- **Frequency Robustness & Evaluation:** We thoroughly evaluate our approach against SOTA baselines on a new Event-CarEnv benchmark based on CarEnv [43], which enables high-throughput Event-RL research. Our results verify that our sequence-based approach outperforms SOTA approaches and exhibits lower susceptibility to variations in observation frequency compared to frame-based methods, maintaining robust performance.
- The code is available at <https://github.com/tgottwald/FLEET>.

2 Related Work

Previous research combining event camera data with machine learning has predominantly focused on supervised learning tasks [6, 20, 25, 42, 48, 59, 63]. Less attention has been given to utilizing event cameras directly as input for reinforcement learning agents [21, 53, 54].

Regarding dimensionality reduction, the supervised approach most closely related to our proposed RL-based feature extractor is that of Sabater *et al.* [42]. They employ a transformer-based architecture for traditional classification tasks, leveraging Perceiver-style cross-attention [26] to reduce sequence length. Their approach uses aggregated event tensors as inputs, patchifies them and adds positional encoding [52] to encode the positional information of the individual patches prior to tokenization. FLEET skips the aggregation and patchification completely and directly works on the information provided by the events.

In the domain of robotic control, Forrai *et al.* [14] tackle an object-catching task using a quadrupedal robot. However, their RL agent does not process event data directly. Rather, classical machine learning methods estimate the object’s impact point. Only these estimated coordinates are passed to the agent, restricting its role strictly to controlling the positioning of the robot.

Vemprala *et al.* [53] utilize a pretrained latent space representation for obstacle avoidance using an autonomous UAV equipped with a simulated event camera. They pretrain a variational autoencoder (VAE) on manually collected data to learn an expressive latent representation of the events. Their encoder processes raw events within a specific time window via a shared multilayer perceptron embedding layer, optionally combining them with a temporal embedding. The decoder then reconstructs an event tensor, minimizing the distance to a tensor aggregated from the raw events. In contrast, our approach eliminates this pretraining phase and the associated need for offline data collection by training end-to-end directly on the control task.

Xu *et al.* [55] apply RL to multimodal inputs, combining RGB images with event camera data. Their model employs a contrastive loss to map fused task-relevant representations and modality-specific noise into a shared latent space, maximizing the distance between these embeddings to explicitly decouple signal from sensor-specific noise. Furthermore, they utilize the off-policy DeepMDP framework [21] for the auxiliary task of next-state and reward prediction. Our method, conversely, operates without the need for auxiliary losses.

Other work, such as Walters *et al.* [54], evaluates RL algorithms on standard, simplified environments adapted to output simulated event camera data.

Additionally, due to the asynchronous nature of event sensor outputs, several studies have explored Spiking Neural Networks (SNNs) for event-based RL agents [30, 55, 57]. However, SNNs remain notoriously difficult to train and often require specialized neuromorphic hardware [5, 10, 38], limiting their general applicability.

3 Fundamentals

In this section we present the foundations of event cameras and Markov decision processes.

3.1 Event Cameras

Standard cameras capture images synchronously at a constant sampling frequency (*e.g.* 30 Hz), recording all pixels simultaneously [16]. In contrast, event

cameras [31] operate asynchronously. They measure per-pixel brightness changes and output an event only when the luminance change exceeds a specific threshold [7, 16]. This generates an asynchronous, high temporal resolution stream of events, denoted as $\mathcal{E} = \{e_k \mid k = 1, \dots, \mathcal{K}\}$. Each event e_k is defined by a tuple $(t_k, \mathbf{u}_k, p_k)$, where t_k is the timestamp of the change, $\mathbf{u}_k = (x_k, y_k)$ specifies the triggered pixels coordinates and p_k indicates the polarity of the event (positive or negative change in luminance) [59].

Transmitting this asynchronous stream instead of images addresses the central bandwidth-latency tradeoff [18] found in time-critical applications. For standard RGB cameras, one must compromise: achieving low latency and high temporal resolution requires high frame rates that saturate bandwidth with highly redundant data, whereas lowering the frame rate to conserve bandwidth increases latency and risks missing critical inter-frame dynamics. However, this asynchronicity introduces significant processing challenges [16], as classical computer vision algorithms are designed for synchronous data formatted in dense grid structures [9, 23, 60]. To bridge this gap, various event stream representations have been developed to make the data compatible with existing vision-based architectures [20, 33, 59, 62].

To process asynchronous events with standard vision architectures, existing methods frequently aggregate the continuous event stream into dense grid structures [20, 55, 63]. A standard approach [20] discretizes events over a sampling window τ into a spatiotemporal tensor of size $H \times W \times T \times 2$. In this format, H and W denote spatial resolution, T represents discrete temporal bins, and the final dimension captures event polarity. By flattening the temporal and polarity dimensions, the data becomes compatible with classical CNNs. However, this forced discretization yields highly sparse tensors that are computationally inefficient to process [37] and tightly couples the computational cost to the sensor’s resolution rather than the actual information content.

3.2 Markov Decision Process

Reinforcement learning is formalized as a Markov decision process (MDP), which is defined as a five-element tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ [3, 49]. $\mathcal{S}$ represents the state space, $\mathcal{A}$ is the action space. The dynamics model $P : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{P}(\mathcal{S})$ describes the transition probability $P(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ of the agent reaching state $\mathbf{s}_{t+1} \in \mathcal{S}$ after executing action $\mathbf{a}_t \in \mathcal{A}$ in its current state $\mathbf{s}_t \in \mathcal{S}$. Typically this definition is extended with the initial state distribution $\rho_0 : \mathcal{P}(\mathcal{S})$, which indicates in which states the agent may start its trajectory. A trajectory $\zeta = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T, \dots)$ describes a consecutive sequence of states and actions. For each transition the agent is awarded with a reward according to the reward function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$. Based on the rewards received and the discount factor γ , the discounted future return is defined as $G(\zeta) = \sum_t \gamma^t r_t$. The agent’s goal is to find an optimal policy π^* which maximizes the expected return J_r :

$$\pi^* = \arg \max_{\pi} J_r(\pi), \text{ with } J_r(\pi) = \mathbb{E}_{\zeta \sim \pi} [G(\zeta)]. \quad (1)$$

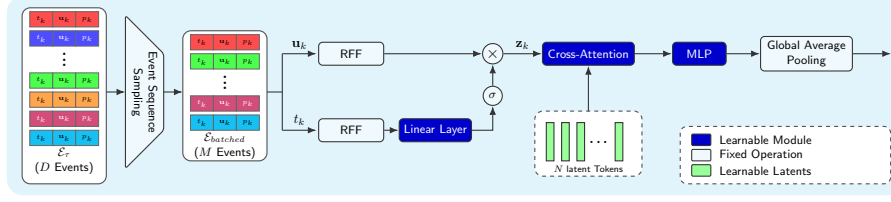


Fig. 2: Overview of our proposed FLEET feature extractor. The number of events within the original sequence $\mathcal{E}_\tau$ is first reduced or padded to $\mathcal{E}_{\text{batched}}$. The temporal and positional component of each event in $\mathcal{E}_{\text{batched}}$ are embedded using shared temporal and spatial random Fourier feature encoders. These embeddings are combined to $\mathbf{z}_k$ using a temporal gating mechanism. The cross-attention module maps the sequence of embedded events to N learnable latent vectors to reduce the dimensionality. The cross-attention is followed by an MLP encoder block. As a last step global average pooling is performed over all latents to form a final representation.

A policy $\pi : \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$ expresses the probability of selecting action $\mathbf{a} \in \mathcal{A}$ while the agent is in state $\mathbf{s} \in \mathcal{S}$. The shorthand notation $\zeta \sim \pi$ indicates that the trajectory ζ is obtained by a random rollout of π .

4 Proposed Method

We propose Feature Learning from Events via Efficient Tokenization (FLEET), a simple yet effective feature extractor architecture. FLEET learns an expressive latent representation directly from raw event camera data for reinforcement learning. Our feature extractor avoids aggregating events into sparse grid structures and requires neither pretraining nor auxiliary tasks.

Instead of aggregating all temporally sorted events within the event stream $\mathcal{E}_\tau$ over a sampling period τ into a sparse tensor, FLEET operates directly on a subsampled, fixed-length sequence, denoted as $\mathcal{E}_{\text{batched}}$. To reduce the original sequence $\mathcal{E}_\tau$ of length D to a fixed length M , $\mathcal{E}_\tau$ is divided into M equally sized bins. The bin edges are defined by the indices $b_k = \lfloor k \frac{D}{M} \rfloor$, $k \in 0, \dots, M-1$, with $b_M = D$. For each bin, a single event at a random index $c_k \sim \mathcal{U}\{b_k, b_{k+1}-1\}$ is selected. If there are fewer than M events, $\mathcal{E}_{\text{batched}}$ is padded with zero values which are masked in subsequent operations. By operating on this fixed-length sequence, our architecture remains closely aligned with the asynchronous paradigm of event cameras. This bypasses the need for the computationally expensive CNNs [20, 55, 63] typically required to overcome the sparsity of grid-based representations.

Event Tokenization and Embedding The temporal $t \in \mathbb{R}$ and spatial components $\mathbf{u} \in \mathbb{R}^2$ of the events are normalized to $[-1, 1]$ and projected into a higher-dimensional embedding space d_{embed} . Standard MLPs suffer from a spectral bias that smooths over high-frequency details from low-dimensional

inputs [50]. To overcome this bias and preserve fine-grained spatio-temporal structures, such as small obstacles or rapid scene dynamics, the coordinates are mapped using random Fourier features (RFF) [39]:

$$\text{RFF}(\mathbf{x}) = [\sin(\pi \mathbf{x} B), \cos(\pi \mathbf{x} B)], \quad (2)$$

where $B \in \mathbb{R}^{d_{\text{input}} \times d_{\text{embed}}/2}$ is sampled from a Gaussian distribution

$$B_{ij} \sim \mathcal{N}(0, \sigma^2). \quad (3)$$

The parameter σ scales the frequency spectrum of the features. Tancik *et al.* have shown that this scale parameter is essential for learning high frequency data like coordinates [50], thus we choose different scalings σ_{temporal} , σ_{spatial} for the temporal and spatial components with the former being chosen to be smaller than the latter.

To effectively fuse the spatiotemporal dynamics, we introduce a temporal gating mechanism. The temporal embedding is linearly projected via learned parameters W_{gate} and $\mathbf{b}_{\text{gate}}$, and passed through a sigmoid activation. This generates a temporal modulation vector that gates the spatial embedding via an element-wise Hadamard product:

$$\mathbf{z}_k = \text{RFF}(\mathbf{u}_k) \otimes \text{sigmoid}(W_{\text{gate}} \text{RFF}(t_k) + \mathbf{b}_{\text{gate}}). \quad (4)$$

Event Sequence Condensation To decouple downstream computational load from the sequence length M , we employ a Perceiver-style [26] condensation module using cross-attention [52]:

$$\text{cross-attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (5)$$

For dimensionality reduction, N learnable latent vectors form the query matrix $Q \in \mathbb{R}^{N \times d_{\text{embed}}}$, while the embedded event sequence $\mathcal{Z}$ provides the keys K and values $V \in \mathbb{R}^{M \times d_{\text{embed}}}$. In practice, we use multi-head cross-attention across h heads, applying a padding mask $M_{\text{mask}} \in \mathbb{R}^{N \times M}$ before the softmax to ignore padded values. This bottleneck compresses the variable-density stream into a dense, fixed-size representation by dynamically attending to the most relevant events.

Backbone and RL Integration The N condensed tokens are processed by a three-layer MLP encoder to capture global scene context and aggregated via global average pooling into a single continuous feature vector. Our FLEET feature extractor passes this representation to a standard Proximal Policy Optimization (PPO) agent [45]. Crucially, while the actor and critic share the feature extractor, we apply a stop-gradient operation to the actor’s inputs [29]. Consequently, the feature extractor’s parameters are updated exclusively via the critic’s value function loss. Driving the representation learning purely through temporal difference updates avoids objective conflicts between the actor and critic, stabilizing the training process.

5 Experiments

In this section, FLEET is evaluated and compared to multiple baseline feature extractors. Our experiments follow best practice [1] and report the interquartile mean (IQM) and 95% confidence interval (CI) using 50 000 bootstrap iterations across ten randomly seeded runs. The parameters for the PPO agent and the individual feature extractors can be found in the Appendix.

5.1 Event-CarEnv

Prior research combining event cameras with reinforcement learning [53, 55] has predominantly relied on the Microsoft AirSim [46] and CARLA [11] simulators. While these platforms excel at photorealistic simulation, their underlying architectures prioritize visual fidelity over the rapid, high-throughput environment interactions required for algorithmic iteration in on-policy RL. To isolate the core mechanics of event-based policy learning, we selected the CarEnv [43] simulator as the foundation for our experiments. CarEnv is explicitly designed to facilitate RL research, providing a highly dynamic, control-focused setting that mirrors the control complexity of established real-world autonomous driving benchmarks, such as Duckietown [36] and the IEEE Bosch Future Mobility Challenge [27]. Our experiments focus on CarEnv’s Racing and Parking scenario. In the Racing scenario, the agent controls a vehicle on a procedurally generated circuit and is rewarded for driving quickly while avoiding track boundaries. This domain randomization frequently introduces sharp turns, forcing the agent to learn a robust policy capable of generalizing across varied track layouts.

The Parking scenario consists of a randomized parking bay where the agent must safely align the vehicle with a target parking position without colliding with the boundaries. Previously, observations within CarEnv have been restricted to set-[8, 13, 44] or lidar-based [22] modalities and their respective feature extractors.

We extend CarEnv with standard RGB and event camera sensors. Racing uses a 30 m frontal lookahead, while Parking centers the vehicle with a 5 m front-and-rear range to enable tight backward and lateral maneuvers. Both sensors use an identical 96×64 pixel spatial resolution, ensuring a fair comparison and matching the low spatial resolution of standard visual RL benchmarks [12, 34]. We model event camera dynamics similarly to [41], estimating asynchronous continuous-time events from discrete RGB renderings via stochastic logarithmic luminance thresholds. See the Appendix for the full formulation.

The event observation $\mathbf{x}_{\text{event}}$ is augmented with a state vector $\mathbf{x}_{\text{state}}$. This comprises velocities v_x and v_y , yaw velocity ω , and steering angle β for Racing, and is limited to $\mathbf{x}_{\text{state}} = (v_x, \beta)$ for Parking [43]. Fig. 3 illustrates a sequence of RGB images alongside aggregated event camera data for the Racing scenario.

5.2 Event Inverted Double Pendulum

To further evaluate FLEET on a classic continuous control benchmark, we adapt the Inverted Double Pendulum task [2, 51]. We integrate the same event generation pipeline as used in Event-CarEnv, configuring the simulated event camera

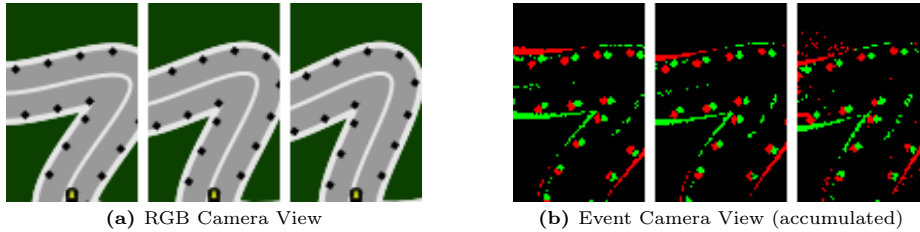


Fig. 3: Sequence of RGB (Fig. 3a) and event (Fig. 3b) camera observations in the Event-CarEnv Racing scenario. Note that the event camera data has been temporarily aggregated into an image format strictly for human interpretability.

to a spatial resolution of 128×128 pixels. Complete details regarding $\Delta t = 0.01$, episode lengths and reward scaling are provided in the Appendix.

5.3 Baselines

Given the scarcity of event-based RL, we compare FLEET against the two most recent reproducible baselines [53, 55] alongside additional baselines inspired by image- and set-based RL:

- **Dense RGB CNN:** A NatureCNN [34] trained on noise-free RGB observations with identical resolution and field-of-view. This is included to relate to the performance of frame-based RL on the problems.
- **Grid-based CNNs:** These methods process events aggregated into RVT [20] with five temporal bins:
 - **NatureCNN [34]:** Adapted from Mnih *et al.* with the first-layer stride reduced from 4 to 1. This prevents the network from skipping sparse, single-pixel events common in tensor-aggregated event data.
 - **ImpalaCNN [12]:** A deeper architecture using residual connections and max pooling, which typically outperforms NatureCNN in pixel-based RL.
 - **DMRCNN [55]:** The event data encoder from Xu *et al.*, utilizing specialized kernels and terminal layer normalization for stabilization.
- **Sequence-based Methods:** These process event streams directly:
 - **LFF-DS [43]:** Based on Schier *et al.* but modified with FLEET’s normalization and temporal gating to isolate the impact of the cross-attention.
 - **eVAE [53]:** A VAE pretrained on a reconstruction task using collected trajectory data. Since we assume that there are no demonstrations or existing policies available we use random policy rollouts. eVAE internally concatenates a stack of the three most recent latent representations.

5.4 Feature Extractor Comparison

We evaluate FLEET in the Event-CarEnv Racing and Parking scenarios ($\Delta t = 0.05, 10^6$ steps) and the Inverted Double Pendulum (IDP) task ($\Delta t = 0.01$,

Table 1: Performance of various feature extractors in combination with PPO. Interquartile means (IQM) and 95% confidence intervals (CI95) of episodic returns are reported for Event-CarEnv Racing/Parking ($\Delta t = 0.05$, 10^6 training steps) and Inverted Double Pendulum ($\Delta t = 0.01$, $5 \cdot 10^6$ steps). **Best** and second-best results are highlighted. FLEET outperforms all baselines across all environments, including a RGB variant on Racing and Parking.

Feature Extractor	Racing (Return $\uparrow$)		Parking (Return $\uparrow$)		Inv. Dbl. Pendulum (Return $\uparrow$)	
	IQM	CI95	IQM	CI95	IQM	CI95
<i>NatureCNN (RGB)</i> [34]	23.89	[12.71, 35.89]	5.01	[4.50, 5.41]	8980.69	[7883.73, 9352.29]
NatureCNN [34]	6.83	[4.41, 11.44]	3.89	[3.50, 4.82]	560.50	[469.09, 1013.37]
ImpalaCNN [12]	4.81	[4.39, 5.93]	<u>5.18</u>	[4.77, 5.59]	<u>1175.57</u>	[184.60, 2178.99]
LFF-DS [43]	5.36	[4.22, 6.46]	0.40	[0.01, 0.81]	848.29	[788.83, 919.08]
eVAE [53]	1.13	[0.81, 1.43]	0.82	[0.78, 0.90]	429.67	[414.25, 441.36]
DMRCNN [55]	<u>12.94</u>	[9.09, 16.99]	4.58	[4.18, 4.96]	470.68	[395.26, 504.51]
FLEET (ours)	46.96	[43.28, 49.03]	5.67	[5.24, 5.84]	6390.41	[5589.50, 7410.47]

$5 \cdot 10^6$ steps). Tab. 1 and Fig. 4 report the final interquartile means, 95% confidence intervals, and training progressions. In the Racing scenario, CNN-based baselines fail to learn viable policies from the sparse tensor representation. The strongest baseline (DMRCNN) achieves only 27% of FLEET’s final performance. Qualitatively, these low return ranges correspond to a degenerate policy where the vehicle simply drives in a straight line and crashes at the first turn. FLEET, however, demonstrates almost monotonic learning and surpasses the RGB variant by 50%. The set-based LFF-DS is also incapable of learning a suitable policy, underscoring the importance of FLEET’s cross-attention module. Furthermore, eVAE fails entirely, likely due to a severe distribution shift, as its pretraining on random rollouts inadequately captures the dynamic state distribution needed for robust control. In the Parking scenario, the performance gap narrows. ImpalaCNN achieves 91% of FLEET’s episodic return, with NatureCNN reaching 68%. We attribute this relative improvement to the smaller spatial distance covered by the sensor’s field of view in the Parking setting. At an identical sensor resolution, the Racing scenario covers three times the vertical distance of the Parking scenario. This reduced spatial scale in the Parking task increases the average event density of the RVT Tensor from 9% to 15%. Because the data is less sparse, the CNN-based feature extractors are less penalized for their architectural limitations. Finally, in the IDP task, FLEET substantially outperforms all event-based competitors (6390.41 vs. <1200). However, the dense RGB baseline scores higher (8980.69). Since IDP features a static camera where the full state remains visible, it inherently favors dense frame-based sensing when bandwidth is unrestricted.

5.5 Resolution Invariance

A core advantage of FLEET is its ability to scale to high-resolution sensors without sacrificing control performance. As demonstrated in Tab. 2 for the Event-

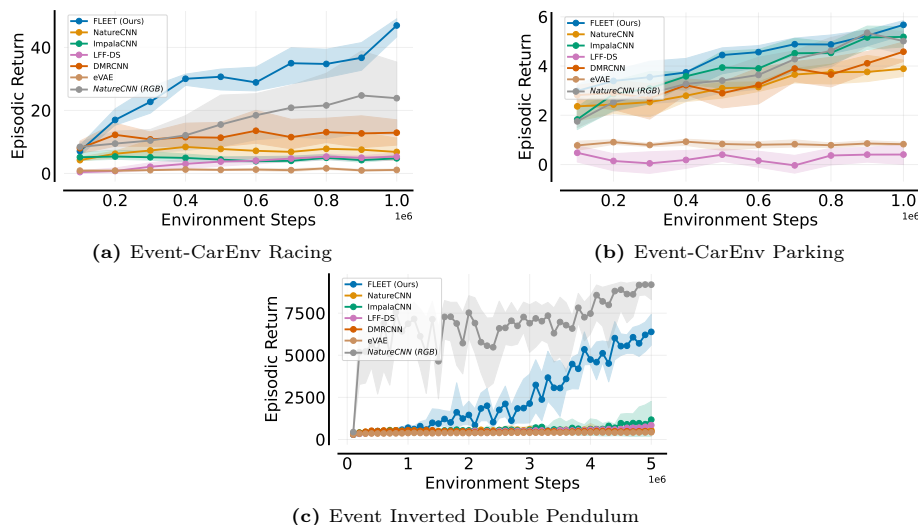


Fig. 4: Progression of the IQM and 95% CI of the episodic return in the Event-CarEnv Racing and Parking ($\Delta t = 0.05$, 10^6 steps) and the Inverted Double Pendulum ($\Delta t = 0.01$, $5 \cdot 10^6$ steps) settings. Agents were evaluated for 10 episodes every 10^5 environment steps, reporting the mean episodic return. In the Racing scenario, CNN-based competitors fail to demonstrate learning progress, though they manage to learn suitable policies in the denser Parking setting. The agents using the eVAE and LFF-DS feature extractor fail to learn a viable policy in any environment. Across all tasks, FLEET shows superior performance to all event-based baseline methods.

CarEnv Racing scenario, FLEET maintains consistently high episodic returns as the spatial resolution of the simulated event camera scales from 96×64 up to 384×256 . By modeling the event stream as a fixed-length point cloud compressed into N latent tokens, the architecture effectively extracts fine-grained dynamics regardless of the native sensor resolution. We contrast this with NatureCNN and DMRCNN, the highest-performing baselines from Sec. 5.4. DMRCNN drops marginally while NatureCNN shows improvement at intermediate resolutions. However, this comes at a computational cost. Because CNNs process grids, their memory and compute requirements scale quadratically. FLEET bypasses both this representation degradation and the computational bottleneck entirely. A detailed analysis of performance and resource scaling is provided in the Appendix.

5.6 Robustness to Input and Control Frequency

High-frequency operation is critical for low-latency control, yet decreasing the observation period Δt reduces the information density and captures less of the environment dynamics per sample. We evaluate robustness by varying frequencies in the Racing scenario ($\Delta t \in \{0.1, 0.05, 0.025\}$). To ensure a fair compari-

Table 2: Interquartile means and 95% confidence intervals of the episodic return of FLEET for different spatial sensor resolutions, evaluated on the Racing scenarios. FLEET’s performance is near constant despite an increase in spatial resolution.

Feature Extractor	96 × 64 (Return ↑)		128 × 96 (Return ↑)		192 × 128 (Return ↑)		384 × 256 (Return ↑)	
	IQM	CI95	IQM	CI95	IQM	CI95	IQM	CI95
NatureCNN	6.83	[4.41, 11.44]	<u>11.43</u>	[9.18, 14.70]	<u>16.42</u>	[11.88, 21.25]	<u>13.21</u>	[10.64, 17.55]
DMRCNN	<u>12.94</u>	[9.09, 16.99]	9.55	[6.90, 13.02]	10.46	[8.08, 13.88]	8.57	[6.41, 12.62]
FLEET (ours)	46.96	[43.28, 49.03]	40.48	[36.49, 44.17]	43.65	[36.85, 49.64]	44.32	[39.89, 49.57]

Table 3: Impact of simulation step size ($\Delta t \in \{0.1, 0.05, 0.025\}$) on episodic return (IQM and 95% CI) in the Racing scenario. Training steps are scaled to $\{0.5 \cdot 10^6, 10^6, 2 \cdot 10^6\}$ respectively to preserve total environment time. FLEET maintains robust performance at higher frequencies compared to CNN-based baseline methods.

Feature Extractor	0.5 · 10 ⁶ steps @ $\Delta t = 0.1$ (Return ↑)			10 ⁶ steps @ $\Delta t = 0.05$ (Return ↑)			2 · 10 ⁶ steps @ $\Delta t = 0.025$ (Return ↑)		
	IQM	CI95		IQM	CI95		IQM	CI95	
NatureCNN [34]	19.02	[16.82, 21.08]		6.83	[4.41, 11.44]		5.87	[4.10, 10.40]	
DMRCNN [55]	<u>21.00</u>	[19.70, 22.62]		<u>12.94</u>	[9.09, 16.99]		<u>6.78</u>	[5.36, 7.71]	
FLEET (ours)	40.33	[38.43, 42.10]		46.96	[43.28, 49.03]		41.92	[34.18, 46.68]	

son, training steps are scaled inversely with Δt to $\{0.5 \cdot 10^6, 10^6, 2 \cdot 10^6\}$ steps to keep the environment time spent on training consistent. Furthermore the discount factor is adjusted to maintain consistent future reward discounting. As shown in Tab. 3 and Fig. 5, the performance of CNN-based NatureCNN and DMRCNN degrades as frequency increases. At $\Delta t = 0.025$, grid-based representations like RVT become highly sparse that standard convolutions fail to process. In contrast, FLEET remains robust across all settings. By operating directly on event sequences, FLEET natively exploits fine-grained temporal dynamics rather than being bottlenecked by sparse spatial tensors. This allows our architecture to maintain high performance even as the individual observation windows shrink at higher frequencies.

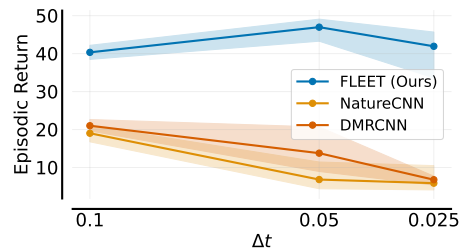


Fig. 5: Episodic return across varying simulation step sizes ($\Delta t = \{0.1, 0.05, 0.025\}$) in the Racing scenario. FLEET remains robust at higher control frequencies, whereas CNN-based architectures degrade.

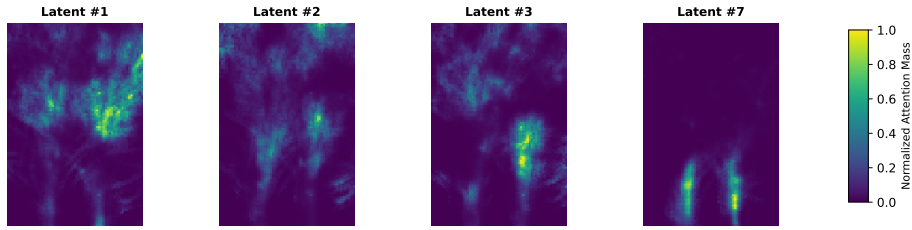


Fig. 6: Normalized spatial attention weights of the event-to-latent mapping during a Racing episode. The activation patterns demonstrate how distinct latents specialize in different regions of the perceptive field. Latent 7 localizes on the track boundaries and cones immediately in front of the ego-vehicle, while Latent 1 attends to broader, distant dynamics in the upper frame. Because the agent maintains a centered racing line, the spatial variance of attended events naturally scales with distance. Latent 3 highlights the agent’s ability to detect track curvature by jointly attending to diagonally opposed regions (*e.g.* upper-left and lower-right), providing context for the steering policy.

5.7 Latent Interpretability

In this section, we investigate the interpretability of the learned latent tokens by visualizing their accumulated attention weights over an Event-CarEnv Racing episode. To ensure visual clarity, the attention mass is normalized independently for each latent. Fig. 6 shows four representative latents, demonstrating how the cross-attention module spatially disentangles the visual field into representations for the agent. Latent 7 acts as a localized feature detector for the ego-vehicle’s immediate proximity. As the agent learns to center the vehicle, this attention mass reliably forms two straight boundary lines. In contrast, Latent 1 captures distant events in the upper visual field, where changes in the camera’s perspective lead to a broader, higher-variance attention distribution. Finally, Latents 2 and 3 integrate both near and distant event clusters. This hierarchical receptive field allows the agent to approximate track curvature, which is a crucial geometric feature for learning a robust steering policy.

5.8 Ablation

Due to page limitations we leave the ablations on different sequence sampling approaches, the number of latent tokens N , the maximum sequence length M and embedding aggregation techniques to the Appendix and focus on the core architectural decisions of FLEET.

Effect of Components: We systematically ablate FLEET’s core modules (Tab. 4). Replacing the cross-attention bottleneck with self-attention increases complexity from $\mathcal{O}(NM)$ to $\mathcal{O}(M^2)$ (where $M \gg N$) and severely degrades performance. Substituting the MLP backbone with a Transformer encoder drops performance by 24%. Unlike prior methods [53], FLEET strongly leverages temporal information. Removing temporal embeddings and gating reduces returns by 22%.

Table 4: Interquartile mean, 95% confidence intervals, and relative change in IQM of the episodic return on Event-CarEnv Racing when removing or replacing components of the FLEET pipeline.

Component	IQM	Return $\uparrow$	
		CI95	Rel. Change
FLEET (Full Pipeline)	46.96	[43.28, 49.03]	-
FLEET (Polarity Embedding)	41.33	[38.07, 45.40]	-11%
FLEET (No Temporal Embedding)	36.44	[33.43, 41.35]	-22%
FLEET (Transformer Backbone)	35.30	[30.12, 41.37]	-24%
FLEET (No Cross-Attention)	33.28	[28.70, 35.97]	-29%

Table 5: Comparison of spatio-temporal embedding techniques for FLEET. Results report the IQM and 95% CI for episodic returns. Random Fourier features achieve the most robust performance.

Spatiotemporal Embedding	IQM	Return $\uparrow$	
		CI95	Rel. Change
Random Fourier Features	46.96	[43.28, 49.03]	-
Learnable LUT Encoding	43.30	[38.07, 47.90]	-7%
Positional Encoding	39.95	[36.19, 43.73]	-14%
Linear Encoding	20.64	[18.89, 23.17]	-56%

Lastly, including polarity embeddings also harms performance, indicating that spatio-temporal data primarily drives success.

Spatio-Temporal Embedding: Following the component analysis, we evaluate the impact of different coordinate projection techniques (Tab. 5). We compare random Fourier features against a learnable look-up table (LUT), standard positional encoding [52], and a simple linear projection. Random Fourier features yield the highest and most consistent returns.

6 Conclusion

In this paper, we introduced FLEET, a novel, end-to-end trainable event camera feature extractor designed explicitly for reinforcement learning. By leveraging random Fourier features and a Perceiver-style cross-attention mechanism, our architecture processes variable-length event sequences and maps them into an expressive, fixed-size latent representation. FLEET bypasses computationally wasteful sparse grid aggregations and inherently decouples the complexity of the model’s backbone from the sensor’s spatial resolution.

We evaluated FLEET against state-of-the-art RL feature extractors across multiple dynamic continuous control tasks, including a novel high-throughput Event-CarEnv autonomous driving benchmark and an Event Inverted Double Pendulum environment. Our extensive empirical evaluations demonstrate that FLEET not only substantially outperforms all baseline methods but also exhibits remarkable robustness to high observation and control frequencies. A setting where traditional grid-based representations severely degrade.

Future work will explore efficient event sampling and filtering techniques to intelligently reduce the raw event sequence length processed by the feature extractor. Furthermore, because current real-world event camera datasets are tailored for supervised learning, a critical next step for practical adoption will be closing the sim-to-real gap and actively addressing the physical deployment of these policies.

Acknowledgements

This work was supported by the MWK of Lower Saxony within Hybrint (VWZN4219) and LCIS (VWZN4704), the Deutsche Forschungsgemeinschaft (DFG) under Germany’s Excellence Strategy within the Cluster of Excellence PhoenixD (EXC2122) and Quantum Frontiers (EXC2123), the European Union under grant agreement no. 101136006 – XTREME.

References

1. Agarwal, R., Schwarzer, M., Castro, P.S., Courville, A.C., Bellemare, M.: Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems* **34**, 29304–29320 (2021)
2. Barto, A.G., Sutton, R.S., Anderson, C.W.: Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics* (5), 834–846 (2012)
3. Bellman, R.: A markovian decision process. *Journal of mathematics and mechanics* pp. 679–684 (1957)
4. Bradbury, J., Frostig, R., Hawkins, P., Johnson, M.J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., Zhang, Q.: JAX: composable transformations of Python+NumPy programs (2018), <http://github.com/jax-ml/jax>
5. Bulzomi, H., Memudu, A.S., Nakano, Y., Martinet, J.: Real-time pedestrian detection at the edge on a fully asynchronous neuromorphic system. In: *Proceedings of the Computer Vision and Pattern Recognition Conference*. pp. 4958–4967 (2025)
6. Cao, H., Zhang, Z., Xia, Y., Li, X., Xia, J., Chen, G., Knoll, A.: Embracing events and frames with hierarchical feature refinement network for object detection. In: *European Conference on Computer Vision*. pp. 161–177. Springer (2024)
7. Chakravarthi, B., Verma, A.A., Daniilidis, K., Fermuller, C., Yang, Y.: Recent event camera innovations: A survey. In: *European Conference on Computer Vision*. pp. 342–376. Springer (2024)
8. Cramer, E., Frauenknecht, B., Sabirov, R., Trimpe, S.: Contextualized hybrid ensemble q-learning: Learning fast with control priors. *arXiv preprint arXiv:2406.19768* (2024)
9. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: *2009 IEEE conference on computer vision and pattern recognition*. pp. 248–255. Ieee (2009)
10. Deng, L., Wu, Y., Hu, X., Liang, L., Ding, Y., Li, G., Zhao, G., Li, P., Xie, Y.: Rethinking the performance comparison between snns and anns. *Neural networks* **121**, 294–307 (2020)
11. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: CARLA: An open urban driving simulator. In: *Proceedings of the 1st Annual Conference on Robot Learning*. pp. 1–16 (2017)
12. Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., et al.: Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In: *International conference on machine learning*. pp. 1407–1416. PMLR (2018)

13. Farkas, P., Becsi, T., Aradi, S.: Residual reinforcement learning enhanced with unsuccessful episode buffer. *IEEE Open Journal of the Computer Society* **7**, 37–48 (2025)
14. Forrai, B., Miki, T., Gehrig, D., Hutter, M., Scaramuzza, D.: Event-based agile object catching with a quadrupedal robot. *arXiv preprint arXiv:2303.17479* (2023)
15. Funk, N., Helmut, E., Chalvatzaki, G., Calandra, R., Peters, J.: Evetac: An event-based optical tactile sensor for robotic manipulation. *IEEE Transactions on Robotics* **40**, 3812–3832 (2024)
16. Gallego, G., Delbrück, T., Orchard, G., Bartolozzi, C., Taba, B., Censi, A., Leutenegger, S., Davison, A.J., Conradt, J., Daniilidis, K., et al.: Event-based vision: A survey. *IEEE transactions on pattern analysis and machine intelligence* **44**(1), 154–180 (2020)
17. Gallego, G., Scaramuzza, D.: Accurate angular velocity estimation with an event camera. *IEEE Robotics and Automation Letters* **2**(2), 632–639 (2017)
18. Gehrig, D., Scaramuzza, D.: Low-latency automotive vision with event cameras. *Nature* **629**(8014), 1034–1040 (2024)
19. Gehrig, M., Aarents, W., Gehrig, D., Scaramuzza, D.: Dsec: A stereo event camera dataset for driving scenarios. *IEEE Robotics and Automation Letters* **6**(3), 4947–4954 (2021)
20. Gehrig, M., Scaramuzza, D.: Recurrent vision transformers for object detection with event cameras. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. pp. 13884–13893 (2023)
21. Gelada, C., Kumar, S., Buckman, J., Nachum, O., Bellemare, M.G.: Deepmdp: Learning continuous latent space models for representation learning. In: *International conference on machine learning*. pp. 2170–2179. PMLR (2019)
22. Gottwald, T., Schier, M., Rosenhahn, B.: Safe resetless reinforcement learning: Enhancing training autonomy with risk-averse agents. In: *European Conference on Computer Vision*. pp. 100–116. Springer (2024)
23. Han, K., Wang, Y., Chen, H., Chen, X., Guo, J., Liu, Z., Tang, Y., Xiao, A., Xu, C., Xu, Y., et al.: A survey on vision transformer. *IEEE transactions on pattern analysis and machine intelligence* **45**(1), 87–110 (2022)
24. Heek, J., Levskaia, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., van Zee, M.: Flax: A neural network library and ecosystem for JAX (2024), <http://github.com/google/flax>
25. Hidalgo-Carrió, J., Gallego, G., Scaramuzza, D.: Event-aided direct sparse odometry. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 5781–5790 (2022)
26. Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., Carreira, J.: Perceiver: General perception with iterative attention. In: *International conference on machine learning*. pp. 4651–4664. PMLR (2021)
27. Kilyen, N.A., Lemnariu, R.F., Mois, G.D., Chen, Y., Morris, B.T., Muntean, I.: The ieee itss and bosch future mobility challenge: A hands-on start to autonomous driving [technical activities]. *IEEE Intelligent Transportation Systems Magazine* **13**(3), 276–282 (2021)
28. Klenk, S., Motzet, M., Koestler, L., Cremers, D.: Deep event visual odometry. In: *2024 International conference on 3D vision (3DV)*. pp. 739–749. IEEE (2024)
29. Kostrikov, I., Yarats, D., Fergus, R.: Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. *arXiv preprint arXiv:2004.13649* (2020)

30. Kou, K., Yang, G., Zhang, W., Yao, Y., Zhou, X.: Event-camera based uav autonomous navigation via spiking reinforcement learning. *Unmanned Systems* pp. 1–16 (2025)
31. Lichtsteiner, P., Posch, C., Delbruck, T.: A 128×128 120 db $15\mu\text{s}$ latency asynchronous temporal contrast vision sensor. *IEEE journal of solid-state circuits* **43**(2), 566–576 (2008)
32. Mahlknecht, F., Gehrig, D., Nash, J., Rockenbauer, F.M., Morrell, B., Delaune, J., Scaramuzza, D.: Exploring event camera-based odometry for planetary robots. *IEEE Robotics and Automation Letters* **7**(4), 8651–8658 (2022)
33. Maqueda, A.I., Loquercio, A., Gallego, G., García, N., Scaramuzza, D.: Event-based vision meets deep learning on steering prediction for self-driving cars. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. pp. 5419–5427 (2018)
34. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. *nature* **518**(7540), 529–533 (2015)
35. Mueggler, E., Gallego, G., Rebecq, H., Scaramuzza, D.: Continuous-time visual-inertial odometry for event cameras. *IEEE Transactions on Robotics* **34**(6), 1425–1440 (2018)
36. Paull, L., Tani, J., Ahn, H., Alonso-Mora, J., Carlone, L., Cap, M., Chen, Y.F., Choi, C., Dusek, J., Fang, Y., et al.: Duckietown: an open, inexpensive and flexible platform for autonomy education and research. In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 1497–1504. IEEE (2017)
37. Perot, E., De Tournemire, P., Nitti, D., Masci, J., Sironi, A.: Learning to detect objects with a 1 megapixel event camera. *Advances in Neural Information Processing Systems* **33**, 16639–16652 (2020)
38. Pfeiffer, M., Pfeil, T.: Deep learning with spiking neurons: Opportunities and challenges. *Frontiers in neuroscience* **12**, 409662 (2018)
39. Rahimi, A., Recht, B.: Random features for large-scale kernel machines. *Advances in neural information processing systems* **20** (2007)
40. Rebecq, H., Gallego, G., Mueggler, E., Scaramuzza, D.: Emvs: Event-based multi-view stereo—3d reconstruction with an event camera in real-time. *International Journal of Computer Vision* **126**(12), 1394–1414 (2018)
41. Rebecq, H., Gehrig, D., Scaramuzza, D.: Esim: an open event camera simulator. In: *Conference on robot learning*. pp. 969–982. PMLR (2018)
42. Sabater, A., Montesano, L., Murillo, A.C.: Event transformer. a sparse-aware solution for efficient event data processing. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 2677–2686 (2022)
43. Schier, M., Reinders, C., Rosenhahn, B.: Learned fourier bases for deep set feature extractors in automotive reinforcement learning. In: *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*. pp. 931–938. IEEE (2023)
44. Schier, M., Schubert, F., Rosenhahn, B.: Explainable reinforcement learning via dynamic mixture policies. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 13530–13536 (2025). <https://doi.org/10.1109/ICRA55743.2025.11127782>
45. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017)
46. Shah, S., Dey, D., Lovett, C., Kapoor, A.: Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In: *Field and service robotics: Results of the 11th international conference*. pp. 621–635. Springer (2017)

47. Shariff, W., Dilmaghani, M.S., Kielty, P., Moustafa, M., Lemley, J., Corcoran, P.: Event cameras in automotive sensing: A review. *IEEE Access* **12**, 51275–51306 (2024)
48. Sun, Z., Messikommer, N., Gehrig, D., Scaramuzza, D.: Ess: Learning event-based semantic segmentation from still images. In: *European Conference on Computer Vision*. pp. 341–357. Springer (2022)
49. Sutton, R.S., Barto, A.G., et al.: *Reinforcement learning: An introduction*, vol. 1. MIT press Cambridge (1998)
50. Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., Ng, R.: Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems* **33**, 7537–7547 (2020)
51. Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al.: Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032* (2024)
52. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
53. Vemprala, S., Mian, S., Kapoor, A.: Representation learning for event-based visuo-motor policies. *Advances in Neural Information Processing Systems* **34**, 4712–4724 (2021)
54. Walters, C., Hadfield, S.: Ceril: Continuous event-based reinforcement learning. *arXiv preprint arXiv:2302.07667* (2023)
55. Xu, H., Peng, P., Tan, G., Li, Y., Xu, X., Tian, Y.: Dmr: Decomposed multi-modality representations for frames and events fusion in visual reinforcement learning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 26508–26518 (2024)
56. Yarats, D., Fergus, R., Lazaric, A., Pinto, L.: Mastering visual continuous control: Improved data-augmented reinforcement learning. *arXiv preprint arXiv:2107.09645* (2021)
57. Zanatta, L., Di Mauro, A., Barchi, F., Bartolini, A., Benini, L., Acquaviva, A.: Directly-trained spiking neural networks for deep reinforcement learning: Energy efficient implementation of event-based obstacle avoidance on a neuromorphic accelerator. *Neurocomputing* **562**, 126885 (2023)
58. Zhang, A., McAllister, R., Calandra, R., Gal, Y., Levine, S.: Learning invariant representations for reinforcement learning without reconstruction. *arXiv preprint arXiv:2006.10742* (2020)
59. Zheng, X., Liu, Y., Lu, Y., Hua, T., Pan, T., Zhang, W., Tao, D., Wang, L.: Deep learning for event-based vision: A comprehensive survey and benchmarks. *arXiv preprint arXiv:2302.08890* (2023)
60. Zhu, L., Liao, B., Zhang, Q., Wang, X., Liu, W., Wang, X.: Vision mamba: Efficient visual representation learning with bidirectional state space model. *arXiv preprint arXiv:2401.09417* (2024)
61. Ziegler, A., Gossard, T., Glover, A., Zell, A.: An event-based perception pipeline for a table tennis robot. *arXiv preprint arXiv:2502.00749* (2025)
62. Zihao Zhu, A., Yuan, L., Chaney, K., Daniilidis, K.: Unsupervised event-based optical flow using motion compensation. In: *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*. pp. 0–0 (2018)
63. Zubic, N., Gehrig, M., Scaramuzza, D.: State space models for event cameras. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 5819–5828 (2024)

Supplementary Material:

FLEET: Token-Based Feature Extraction for Event Camera-based Reinforcement Learning

A Experiments

A.1 Experimental Setup

All experiments were performed using Proximal Policy Optimization [45] with a learning rate of $\lambda = 0.0003$, employing separate MLP architectures for the actor and critic, each consisting of two hidden layers of 256 units. The event data is processed by the respective feature extractor while the state vector of the environment is always processed by a single linear layer followed by a relu activation. The respective latent representations are concatenated prior to being passed to the agent. Following [29], the feature extractors are shared between actor and critic. Unless specified otherwise, the experiments were conducted without framestacking. The agent was trained for 10^6 steps with evaluation on 10 environment instances taking place every 10^5 steps.

All methods were implemented in JAX [4] using the Flax [24] framework.

A.2 Hyperparameters

Tab. 6 and Tab. 7 provide the hyperparameters used in our experiments unless specified otherwise.

Table 6: FLEET hyperparameters used in all of our experiments.

Hyperparameter	Value
FLEET	
Event Sequence Length (M)	2048
Embedding Dimension (d_{embed})	128
σ_{temporal}	5.0
σ_{spatial}	20.0
Number of Latent Tokens (N)	32
Attention Heads (h)	4
Encoder MLP Widths	$(2 \cdot d_{\text{embed}}, 2 \cdot d_{\text{embed}}, d_{\text{embed}})$

Table 7: Hyperparameters for the baseline feature extractors used in our experiments.

Hyperparameter	Value
NatureCNN	
Conv. Architecture (Channels, Kernel, Stride)	(32, 8, 1), (64, 4, 2), (64, 3, 1)
Output Dimension	512
ImpalaCNN	
Conv. Arch. (Features, Kernel, Stride, ResBlocks)	(16, 3, 1, 2), (32, 3, 1, 2), (32, 3, 1, 2)
Output Dimension	256
DMRCNN	
Conv. Architecture (Channels, Kernel, Stride)	(64, 5, 2), (128, 3, 2), (256, 3, 2), (256, 3, 2)
Output Dimension	3072
LFF-DS	
Item Encoder Units	(128, 128)
Set Encoder Units	(256, 256, 128)
eVAE	
Latent Dimension	8
ECN Layers	(64, 128, 1024)
Encoder Layers	(256, 16)
Decoder Layers	(128, 256)

A.3 Event-CarEnv

Event Generation We model the event camera dynamics of Event-CarEnv by converting rendered RGB frames first to grayscale frames I and then into a logarithmic luminance space $L(\mathbf{u}, t) = \ln(I(\mathbf{u}, t) + \epsilon)$. Events are triggered asynchronously at pixel $\mathbf{u}$ whenever the log-intensity deviation ΔL from the stored reference state L_{ref} exceeds the specific contrast threshold for that polarity. We define separate thresholds ϑ_{on} for increasing intensity ($\Delta L > 0$) and ϑ_{off} for decreasing intensity ($\Delta L < 0$). We adopt the calibrated parameter values from ESIM [41], where thresholds are sampled from normal distributions with means $\mu_{\text{on}} = 0.55$, $\mu_{\text{off}} = 0.41$ and standard deviation $\sigma = 0.021$.

The polarity of the event is defined as $p = \text{sgn}(\Delta L)$. Let the effective threshold be $\vartheta = \vartheta_{\text{on}}$ if $p = 1$ and $\vartheta = \vartheta_{\text{off}}$ if $p = -1$. Given a discrete simulation timestep Δt , the number of triggered events is $\mathcal{N} = \lfloor |\Delta L| / \vartheta \rfloor$. Following the interpolation approach of ESIM, we approximate the continuous-time nature of event generation within discrete rendering steps by estimating the timestamp t_k for the k -th event:

$$t_k = t_{\text{start}} + \left(\frac{k \cdot \vartheta \cdot p}{\Delta L} \right) \Delta t, \quad k \in \{1, \dots, \mathcal{N}\}. \quad (6)$$

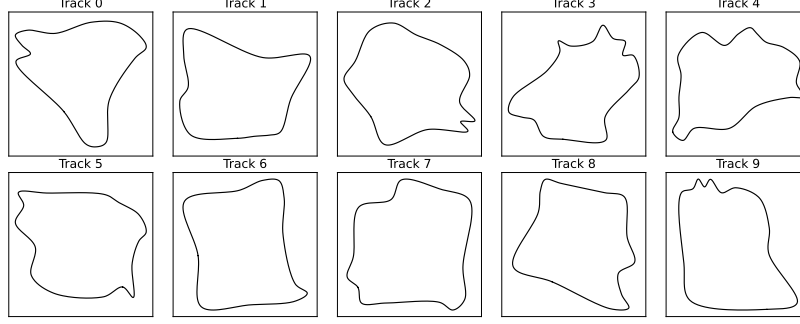


Fig. 7: Course of the ten tracks used during evaluation in the Event-CarEnv Racing setting. The tracks cover different degrees of difficulty from easy (Track 6) to very hard (Track 3).

To ensure sensor realism, timestamps are constrained by a hardware refractory period τ_{ref} , such that $t_k \leftarrow \max(t_k, t_{k-1} + \tau_{\text{ref}})$, and the reference state L_{ref} is updated incrementally by $N \cdot \vartheta \cdot p$ to preserve sub-threshold residuals across simulation frames.

Racing For a better understanding of the complexity of the task we provide the courses of the tracks used during evaluation the Racing setting in Fig. 7. The tracks differ in difficulty, with some requiring the agent to perform multiple sharp turns in short succession (*e.g.* Tracks 0, 2 and 9) and others being easier due to a more simplistic track layout (*e.g.* Track 6).

Furthermore, Fig. 8 visualizes a three frame sequence of the vehicle driving on a relatively straight segment of the track.

The reward function remains identical to [43]:

$$r(\mathbf{s}, \mathbf{a}, \mathbf{s}') = -\mathbb{1}_{\text{fail}} - 0.2 \cdot \mathbb{1}_{\text{collision}} + 0.01 \cdot \mathbf{n}_{\text{track}} \cdot \mathbf{v}_{\text{ego}}, \quad (7)$$

with $\mathbf{v}_{\text{ego}}$ being the vehicles velocity vector which is projected onto the tracks forward direction $\mathbf{n}_{\text{track}}$.

Parking Fig. 9 shows a sequence of frames from the RGB camera and accumulated event camera data for the Event-CarEnv Parking scenario.

Similar to the Racing scenario the reward function is kept unchanged from [43]:

$$r(\mathbf{s}, \mathbf{a}, \mathbf{s}') = -\mathbb{1}_{\text{collision}} + c \cdot r_{\text{pose}}(\mathbf{s}, \mathbf{a}, \mathbf{s}'), \quad (8)$$

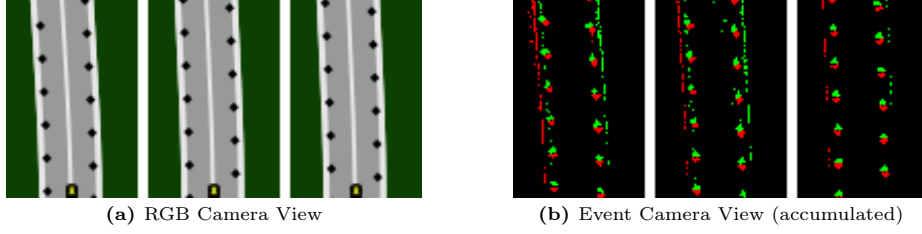


Fig. 8: Additional Sequence of RGB (Fig. 3a) and event (Fig. 3b) camera observations of a relatively straight section in the Event-CarEnv Racing scenario. Note that the event camera data has been temporarily aggregated into an image format strictly for human interpretability.

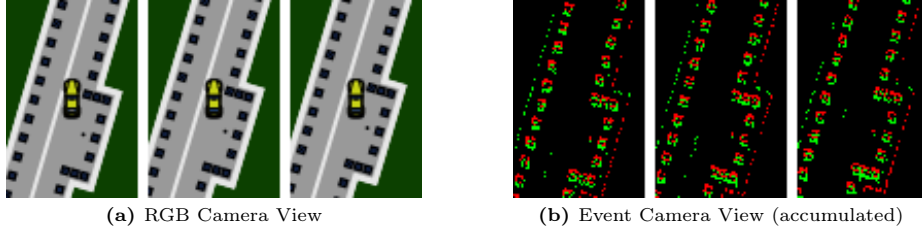


Fig. 9: Sequence of RGB (Fig. 3a) and event (Fig. 3b) camera observations in the Event-CarEnv Parking scenario. Note that the event camera data has been temporarily aggregated into an image format strictly for human interpretability.

with

$$r_{\text{pose}}(\mathbf{s}, \mathbf{a}, \mathbf{s}') = \max\left(0, 1 - \frac{|x - u|}{x_{\max}}\right) \quad (9)$$

$$\cdot \max\left(0, 1 - \frac{|y - v|}{y_{\max}}\right) \cdot \max(0, \cos(\alpha - \kappa)). \quad (10)$$

The second reward term r_{pose} penalizes the current vehicle pose (x, y, α) deviating from the target pose (u, v, κ) . The scaling coefficients remain unchanged at $c = 0.05$, $x_{\max} = 40\text{m}$ and $y_{\max} = 5\text{m}$.

A.4 Event Inverted Double Pendulum

For the Inverted Double Pendulum environment the default frame-skipping mechanism is removed, resulting in a dense simulation step size of $\Delta t = 0.01$ instead of $\Delta t = 0.05$. To ensure comparability between the modified version of Inverted Double Pendulum used in the evaluation and the standard version, the discount factor, episode length and number of training steps is adjusted. The maximum episode length is proportionally increased to 5000 interactions and the total number of environment steps to $5 \cdot 10^6$. The reward is augmented with a scaling

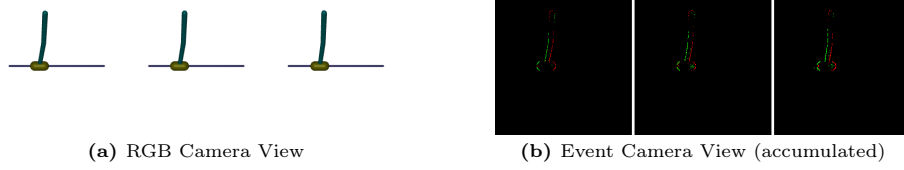


Fig. 10: Sequence of RGB (Fig. 10a) and event (Fig. 10b) camera observations in the Inverted Double Pendulum. Note that the event camera data has been temporarily aggregated into an image format strictly for human interpretability.

factor c_{scaling} based on Δt , keeping the episodic returns in the same range as the default setting. Thus the reward function can be formulated as:

$$r(\mathbf{s}, \mathbf{a}, \mathbf{s}') = c_{\text{scaling}} \cdot (10 \cdot \mathbb{I}_{\text{alive}} - (0.01 \cdot x_{\text{tip}}^2 + (y_{\text{tip}} - 2)^2) - (10^{-3} \cdot \omega_1^2 + 5 \cdot 10^{-3} \cdot \omega_2^2)), \quad (11)$$

where $c_{\text{scaling}} = \Delta t / 0.05$, x_{tip} and y_{tip} are the positions of the free swinging tip of the pendulum and ω_1 and ω_2 are the hinge’s angular velocities.

Furthermore, we increase the discount factor to account for the increased episode length. In addition to the visual observation of the environment, the observation space is augmented with the velocity of the cart and angular velocities of the pendulum [51].

In contrast to the Event-CarEnv scenarios, the Inverted Double Pendulum setting keeps the cameras position static as shown in Fig. 10. While in Racing and Parking the sensors only capture a part of the track, in Inverted Double Pendulum the complete environment is visible at all times.

B Memory Usage and Performance

A core advantage of the FLEET architecture over standard CNN-based approaches is its ability to maintain superior performance as spatial resolution scales up. While baseline methods like NatureCNN can also yield increased episodic returns at higher spatial resolutions, they do so at a severe computational penalty. To quantify this bottleneck, the computational load in floating point operations (FLOPs), the execution time, and the VRAM usage per minibatch (5120 transitions) were analyzed across four spatial sensor resolutions (96×64 , 128×96 , 192×128 , and 384×256) in Tab. 8, Tab. 9, and Fig. 11. All experiments were conducted on a dedicated compute node equipped with an NVIDIA RTX 3090 GPU, 4 CPU threads, and 32 GB of system RAM. At the lowest resolution of 96×64 , FLEET requires 83.28 GFLOPs, which is initially higher than DMRCNN at 47.11 GFLOPs. However, despite this higher theoretical FLOP count, FLEET exhibits a significantly lower inference execution time of 11.76 ms compared to DMRCNN’s 81.48 ms and NatureCNN’s 286.63 ms. This discrepancy demonstrates that FLEET relies on highly parallelizable, GPU-friendly operations, such as dense matrix multiplications in its attention mechanisms, which maps efficiently to modern hardware.. After only a

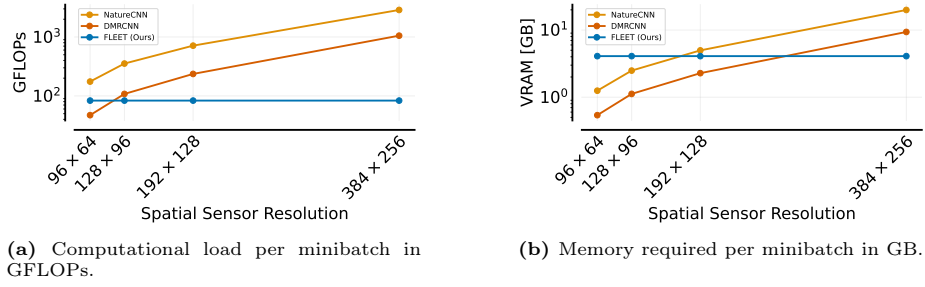


Fig. 11: Comparison of computational cost (Fig. 11a) and memory consumption (Fig. 11b) per minibatch (5120 transitions) for NatureCNN, DMRCNN, and FLEET across increasing spatial resolutions.

slight increase in spatial resolution to 128×96 , FLEET immediately demands the lowest computational load among all tested models. At this 128×96 resolution, FLEET’s constant 83.28 GFLOPs drops below DMRCNN’s 108.05 GFLOPs and requires roughly 23% of the compute needed by NatureCNN’s 353.93 GFLOPs. NatureCNN’s high computational load can be explained by the choice to decrease the stride to 1 instead of 4 to prevent the feature extractor from skipping single-pixel events. As resolutions scale further, this efficiency gap widens exponentially because CNN architectures must process dense spatial grids, causing their compute and memory requirements to scale quadratically. FLEET completely bypasses this representation degradation and computational bottleneck by compressing the variable event stream into a fixed number of latent tokens via cross-attention. Consequently, FLEET maintains a strictly constant load of 83.28 GFLOPs and a fixed 4.09 GB VRAM footprint across all evaluated resolutions. At the maximum evaluated resolution of 384×256 , FLEET uses just 8% of the 1053.09 GFLOPs required by DMRCNN, and a mere 3% of the 2876.33 GFLOPs demanded by NatureCNN. In terms of actual inference speed, this translates to FLEET executing over 310 times faster than DMRCNN (3690.64 ms) and over 400 times faster than NatureCNN (4778.59 ms) per minibatch at this resolution. This completely flat scaling curve should enable our architecture to efficiently leverage modern, high-definition event sensors without the immense overhead associated with grid-based methods.

C Ablations

This section includes ablations of our feature extractor that could not be included in the main submission due to page constraints.

Sequence Sampling Technique: We evaluate the impact of our sequence sampling strategy by comparing it against a temporal sampling baseline that simply selects the most recent M events. As shown in Table 10, replacing FLEET’s stratified sampling with this temporal heuristic results in a 21% drop in episodic return.

Table 8: Comparison of computational cost (GFLOPs) and memory consumption (VRAM in GB) per minibatch (5120 transitions) for NatureCNN, DMRCNN, and FLEET across increasing spatial resolutions. Because FLEET directly processes a fixed-length sequence of events rather than sparse grids, its computational load and memory footprint remain perfectly constant, effectively decoupling inference cost from the sensor’s spatial resolution.

Feature Extractor	96 × 64		128 × 96		192 × 128		384 × 256	
	GFLOPs(↓)	VRAM [GB](↓)	GFLOPs(↓)	VRAM [GB](↓)	GFLOPs(↓)	VRAM [GB](↓)	GFLOPs(↓)	VRAM [GB](↓)
NatureCNN	175.11	<u>1.25</u>	353.93	<u>2.49</u>	712.84	4.98	2876.33	19.92
DMRCNN	47.11	0.54	<u>108.05</u>	1.12	<u>236.07</u>	2.28	<u>1053.09</u>	<u>9.37</u>
FLEET (ours)	<u>83.28</u>	4.09	83.28	4.09	83.28	<u>4.09</u>	83.28	4.09

Table 9: Inference execution time per minibatch for NatureCNN, DMRCNN and FLEET across increasing spatial resolutions. FLEET leverages GPU-friendly operations, leading to lower execution times than it’s competitors.

Feature Extractor	96 × 64	128 × 96	192 × 128	384 × 256
	Exec. Time [ms](↓)	Exec. Time [ms](↓)	Exec. Time [ms](↓)	Exec. Time [ms](↓)
NatureCNN	286.63	577.67	1165.83	4778.59
DMRCNN	<u>81.48</u>	<u>260.72</u>	<u>644.07</u>	<u>3690.64</u>
FLEET (ours)	11.76	11.76	11.76	11.76

While dense scenes inevitably require discarding events to satisfy sequence length constraints, our approach distributes this reduction uniformly across the entire sampling period. This strategy successfully preserves the overall spatio-temporal geometry of the scene and avoids the severe temporal blindspots that occur when truncating earlier events. Maintaining this structural integrity provides a critical advantage for the agent, as real event camera bursts are highly asynchronous and rarely linearly spaced in time.

Table 10: Interquartile mean, 95% confidence intervals, and relative change in IQM of the episodic return when using temporal sampling (choose last M events) when compared to the sampling approach used in FLEET.

Sequence Sampling Technique	Return ↑		
	IQM	CI95	Rel. Change
FLEET (ours)	46.96	[43.28, 49.03]	-
Temporal Sampling	37.02	[34.68, 39.89]	-21%

Matched Sequence Baseline: To isolate the performance gains of our proposed architecture from the event sequence sampling, we evaluated a parameter-matched MLP sequence baseline. This baseline processes identical sampled event sequences but replaces the Random Fourier Features with standard linear embeddings and entirely removes the cross-attention bottleneck. As shown in Table 11, replacing FLEET with this baseline results in a 35% drop in episodic return. While an alternative Transformer-based sequence encoder could theoretically be applied, its computational complexity scales quadratically with the number of events in the absence of our bottleneck mechanism, which undermines the efficiency required for high-frequency control. These results demonstrate that while our sampling strategy provides a strong foundation, FLEET’s specific architectural innovations account for approximately one-third of the overall performance improvement.

Table 11: Interquartile mean, 95% confidence intervals, and relative change in IQM of the episodic return comparing the full FLEET architecture to a parameter-matched MLP sequence baseline. This comparison isolates the performance gains provided by the Random Fourier Features and the cross-attention bottleneck from the inherent benefits of the underlying sequence sampling strategy.

Sequence Sampling Technique	IQM	Return $\uparrow$	
		CI95	Rel. Change
FLEET (ours)	46.96	[43.28, 49.03]	-
Matched Sequence Baseline	30.06	[26.13, 36.12]	-35%

Number of Latent Tokens N : As demonstrated before, the cross-attention mechanism successfully maps raw events to semantically consistent latent tokens. Therefore, we investigate how varying the total number of these latent tokens N impacts the episodic return. We evaluate FLEET across $N \in \{8, 16, 32, 64, 128\}$, with the results visualized in Fig. 12a. The experiments reveal that decreasing the number of latent tokens below $N = 32$ leads to a steady decline in episodic return. Conversely, increasing the token count beyond $N = 32$ not only lowers the average return but also widens the confidence intervals.

We hypothesize that with too few latent tokens, the feature extractor loses expressivity, forcing the model to compress semantically distinct events into the same token. On the other hand, an excess of tokens causes the model to distribute a single semantic event across multiple latents. This introduces ambiguity during the learning process, which manifests as higher variance in the confidence intervals.

Event Sequence Length M : Since the number of events D within a sampling period is variable due to the asynchronous nature of the sensor, the raw event

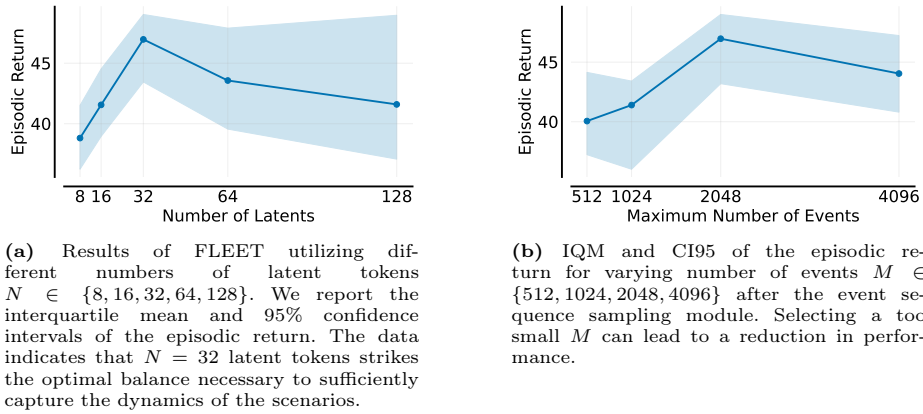


Fig. 12: Hyperparameter study on the Event-CarEnv Racing scenario for the number of latents N in the cross-attention module (Fig. 12a) and the maximum event sequence length M output by the event sequence sampling module (Fig. 12b).

Table 12: IQM and CI95 of FLEET on Event-CarEnv Racing with varying number of latents N in the cross-attention module. Both too many and not enough latents lead to performance degradation.

Feature Extractor	$N = 8$ (Return $\uparrow$)		$N = 16$ (Return $\uparrow$)		$N = 32$ (Return $\uparrow$)		$N = 64$ (Return $\uparrow$)		$N = 128$ (Return $\uparrow$)	
	IQM	CI95	IQM	CI95	IQM	CI95	IQM	CI95	IQM	CI95
FLEET (ours)	38.81	[35.99, 41.09]	41.55	[38.94, 44.57]	46.96	[43.28, 49.03]	43.56	[39.51, 47.71]	41.59	[37.11, 48.52]

sequence can be difficult to process. CNN-based approaches avoid this by aggregating the data into a grid which can result in a loss of information. FLEET works directly on the event sequence data. However, the size of the attention matrix of the cross-attention module is dependent on the number of events. Furthermore, choosing JAX for our implementation requires fixed input sizes to leverage JAX’s acceleration. Thus FLEET preprocesses the variable length raw event sequence to a fixed-length sequence. For this, the number of events is padded with zero values if there are fewer than M events and sampled if there are more than M events.

In this experiment, we investigate the impact of different sequence lengths M in the Event-CarEnv Racing scenario. The scenario has a mean raw event count of 3741 events per observation step. Tab. 13 and Fig. 12b show that reducing M too much can lead to a drop in episodic return as too few relevant events are captured.

Embedding Aggregation Technique: FLEET performs element-wise gating of the spatial embedding with a linear projection of the temporal embedding. In this

Table 13: Interquartile mean (IQM) and 95% confidence intervals (CI95) of the episodic return for the FLEET architecture on the Event-CarEnv Racing scenario, evaluated across varying event sequence lengths M . The results illustrate the impact of the sequence length sampled prior to the cross-attention bottleneck, demonstrating how excessively small values of M can constrain the network’s performance, while still performing better than CNN-based approaches.

Feature Extractor	$M = 512$ (Return $\uparrow$)		$M = 1024$ (Return $\uparrow$)		$M = 2048$ (Return $\uparrow$)		$M = 4096$ (Return $\uparrow$)	
	IQM	CI95	IQM	CI95	IQM	CI95	IQM	CI95
FLEET (ours)	40.05	[37.23, 44.14]	41.39	[36.00, 43.41]	46.96	[43.28, 49.03]	44.04	[40.80, 47.23]

ablation we replace the temporal gating with other aggregation techniques. We perform element-wise addition and multiplication. Furthermore, we also try concatenating the different embeddings and projecting them back to the original embedding dimension afterwards. The results can be found in Tab. 14. Using the temporal gating mechanism instead of the other approaches results in higher episodic returns, strengthening the motivation to use temporal gating in our approach.

Table 14: Interquartile mean, 95% confidence intervals, and relative change in IQM of the episodic return when replacing the temporal gating embedding aggregation strategy with element-wise addition or multiplication and concatenation. Temporal gating shows the highest episodic returns.

Aggregation Technique	Return $\uparrow$		
	IQM	CI95	Rel. Change
Temporal Gating	46.96	[43.28, 49.03]	-
Addition	40.03	[36.47, 44.25]	-14%
Multiplication	31.83	[27.39, 35.80]	-32%
Concatenation	39.74	[34.35, 45.91]	-15%